Práctica Semana 4 – Algoritmos Avanzados de Ordenamiento No Comparativo

OBJETIVO

- Comprender e implementar algoritmos de ordenamiento no comparativos: **Counting Sort, Radix Sort, Bucket Sort y Timsort**.
- Analizar su complejidad temporal y espacial comparándolos con algoritmos comparativos (QuickSort, MergeSort).
- Aplicar estos algoritmos en la solución de un problema real de **clasificación masiva de datos numéricos**.

MARCO TEÓRICO

1. Ordenamiento No Comparativo

Los algoritmos de ordenamiento no comparativos no realizan comparaciones directas entre pares de elementos, a diferencia de algoritmos clásicos como **QuickSort** o **MergeSort**. En lugar de eso, se apoyan en propiedades adicionales de los datos (como el rango, la distribución o la representación en dígitos) para ordenar más rápido. Su ventaja radica en que, bajo ciertas condiciones, pueden lograr complejidades **sub-lineales en comparación con n log n**, lo que los hace ideales para **grandes volúmenes de datos con dominios limitados**.

2. Counting Sort

- **Concepto**: Se basa en contar cuántas veces aparece cada elemento en el conjunto de datos. Utiliza un arreglo auxiliar (count[]) para almacenar las frecuencias y reconstruir la lista ordenada.
- **Complejidad**:
  - Tiempo: **O(n + k)**, donde *n* es el tamaño de la entrada y *k* el valor máximo posible en el arreglo.
  - Espacio: **O(n + k)** (se requiere memoria adicional).
- **Aplicaciones**:
  - Ordenamiento de **notas de exámenes**, **clasificación de edades**, o cualquier escenario donde los valores estén en un rango pequeño y definido.
- **Limitaciones**:
  - Ineficiente cuando el rango de datos (*k*) es mucho mayor que el número de elementos (*n*).

3. Radix Sort

- **Concepto**: Ordena los elementos procesando cada dígito, desde el menos significativo (LSD) o el más significativo (MSD), aplicando **Counting Sort** como subrutina.
- **Complejidad**:
  - Tiempo: **O(d · (n + k))**, con *d* número de dígitos y *k* rango de valores de cada dígito.
  - Espacio: **O(n + k)**.
- **Características**:
  - Estable (conserva el orden de elementos iguales).
  - Eficiente para ordenar números enteros grandes o cadenas de caracteres.
- **Aplicaciones**:

    - Procesamiento de **números telefónicos**, **códigos postales**, **placas de vehículos**.
- **Limitaciones**:
    - Requiere subrutinas estables.
    - Puede ser más costoso en problemas con números de gran longitud en dígitos.

4. Bucket Sort

- **Concepto**: Divide los elementos en intervalos o "cubetas" según un criterio de rango. Cada cubeta se ordena individualmente (normalmente con Insertion Sort) y luego se concatenan.
- **Complejidad**:
    - Mejor caso: **O(n + k)** si la distribución es uniforme.
    - Peor caso: **O(n^2)** si todos los elementos caen en la misma cubeta.
- **Aplicaciones**:
    - Ordenamiento de **números en punto flotante** entre 0 y 1.
    - Datos con **distribución uniforme** en intervalos conocidos (ejemplo: sensor de temperaturas).
- **Limitaciones**:
    - Depende mucho de la **función de dispersión** y de la distribución de los datos.

5. Timsort

- **Concepto**: Es un algoritmo híbrido entre **MergeSort** e **InsertionSort**, diseñado para rendir bien con datos reales.
    - Divide el arreglo en pequeños bloques llamados **runs**.
    - Ordena cada run con Insertion Sort.
    - Combina los runs con MergeSort adaptativo.
- **Complejidad**:
    - Mejor caso: **O(n)** si el arreglo ya está casi ordenado.
    - Peor caso: **O(n log n)**.
- **Características**:
    - Estable.
    - Aprovecha patrones de orden ya existentes en los datos.
- **Aplicaciones**:
    - Es el algoritmo por defecto en **Python (sorted())** y en **Java** para listas de objetos.
    - Muy usado en **procesamiento de grandes volúmenes de datos heterogéneos**.
- **Limitaciones**:
    - Requiere memoria adicional para las fusiones.

6. Comparación General

Algoritmo	Complejidad Promedio	Espacio Extra	Estabilidad	Ideal para...
Counting Sort	O(n + k)	O(n + k)	Sí	Valores enteros en rango limitado
Radix Sort	O(d·(n + k))	O(n + k)	Sí	Enteros grandes, cadenas, claves
Bucket Sort	O(n + k) (mejor caso)	O(n + k)	Sí	Datos distribuidos uniformemente

Timsort	O(n log n)	O(n)	Sí	Listas reales con patrones preexistentes

CÓDIGO DE LA PRÁCTICA

Ejemplo en Python: Radix Sort + Timsort

```python
# Radix Sort
def counting_sort(arr, exp):
    n = len(arr)
    output = [0] * n
    count = [0] * 10

    for i in range(n):
        index = (arr[i] // exp) % 10
        count[index] += 1

    for i in range(1, 10):
        count[i] += count[i - 1]

    i = n - 1
    while i >= 0:
        index = (arr[i] // exp) % 10
        output[count[index] - 1] = arr[i]
        count[index] -= 1
        i -= 1

    for i in range(n):
        arr[i] = output[i]

def radix_sort(arr):
    max_num = max(arr)
    exp = 1
    while max_num // exp > 0:
        counting_sort(arr, exp)
        exp *= 10

# Timsort (Python ya lo implementa con sorted())
arr = [329, 457, 657, 839, 436, 720, 355]
print("Original:", arr)

radix_sort(arr)
print("Radix Sort:", arr)
print("Timsort (built-in):", sorted(arr))
```

Ejemplo en C++: Counting Sort

```cpp
#include <iostream>
#include <vector>
using namespace std;

void countingSort(vector<int>& arr, int maxVal) {
    vector<int> count(maxVal + 1, 0);
    vector<int> output(arr.size());
```

```
    for (int num : arr) count[num]++;
    for (int i = 1; i <= maxVal; i++) count[i] += count[i - 1];
    for (int i = arr.size() - 1; i >= 0; i--) {
        output[count[arr[i]] - 1] = arr[i];
        count[arr[i]]--;
    }
    arr = output;
}

int main() {
    vector<int> arr = {5, 2, 1, 7, 3, 2, 5, 0};
    countingSort(arr, 7);
    cout << "Resultado: ";
    for (int num : arr) cout << num << " ";
    return 0;
}
```

DESARROLLO (1 hora máx.)

1. Implementar en Python y C++ **dos algoritmos de ordenamiento no comparativo** de su elección.
2. Comparar los tiempos de ejecución frente a **QuickSort/MergeSort** con arreglos de **10^5 números aleatorios**.
3. Analizar casos donde **Counting Sort y Radix Sort** son más eficientes que los algoritmos comparativos.

TRABAJO DE INVESTIGACIÓN

- Investigar el **funcionamiento interno de Timsort** y explicar por qué es utilizado como estándar en Python.
- Comparar el rendimiento de **Bucket Sort** aplicado a datos distribuidos uniformemente vs. distribuciones sesgadas.
- Documentar al menos **3 casos reales** donde se usan algoritmos no comparativos (ej: sistemas de facturación, clasificación de imágenes, big data).

RESULTADOS ESPERADOS

- Programas funcionales en Python y C++.
- Tabla comparativa de tiempos y memoria usada.
- Informe breve (máx. 2 páginas) con:
    - Explicación teórica de cada algoritmo.
    - Comparación de eficiencia con ejemplos prácticos.
    - Conclusiones sobre el algoritmo más adecuado según el contexto.